

SQL Server on Kubernetes

Redefining Modern Database
Infrastructure

Nader Sharara





Thank you to our sponsors.
Please say hello to them and find
out how they can help you.



Who am I?



Database Administrator & Architect with ~15 years of experience.

Specialized in SQL Server across on-prem and cloud environments.

Worked across DBA, Database Engineering Management, and Cloud Database Architecture roles.

Microsoft
CERTIFIED
IT Professional
Database Administrator
2008

Microsoft
CERTIFIED
Solutions Expert
Data Platform



sqlspark.com



[nader-sharara-db](https://www.linkedin.com/in/nader-sharara-db)

Currently building SQL Server cloud managed services at STACKIT.

Sharing technical insights at sqlspark.com.

Today's Roadmap

- Why containerized SQL Server matters.
- Core Kubernetes concepts for database workloads.
- SQL Server building blocks on Kubernetes: StatefulSets, storage, and operators.
- High availability on Kubernetes: self-healing, Availability Groups, and failover coordination.
- Key takeaways, demo, and questions.

Why Containerizing SQL Server?

Containers bring **speed, consistency, and flexibility** to how we build, run, and scale SQL Server.



Speed & Agility

Provision SQL Server environments in **minutes** instead of waiting hours or days.



Consistency

Use the same image and configuration across **development, testing, and production**.

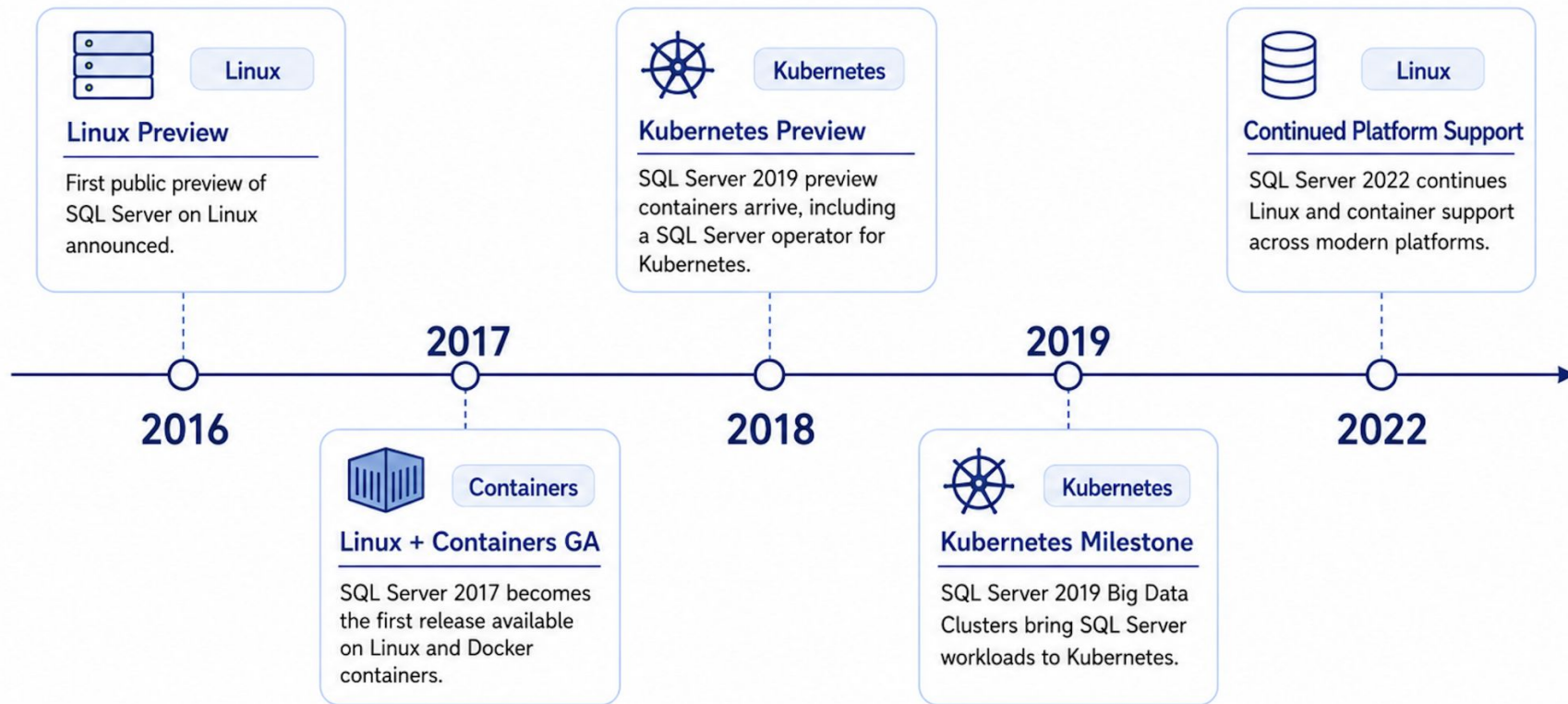


Portability & Density

Run SQL Server across different environments with better **flexibility** and **resource efficiency**.



SQL Server on Linux, Containers & Kubernetes: Key Milestones



Why Kubernetes Matters for SQL Server?



At container scale, manual operations break down

Running one SQL Server container is easy.
Running many environments consistently is much harder.



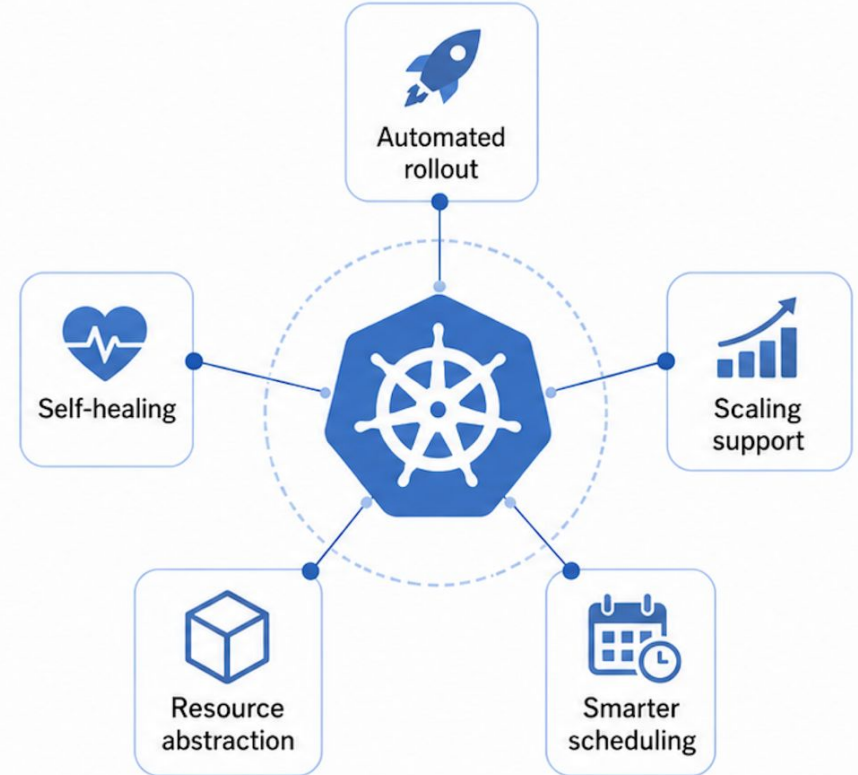
Kubernetes provides the orchestration layer

It automates how containerized workloads are deployed, managed, and recovered.

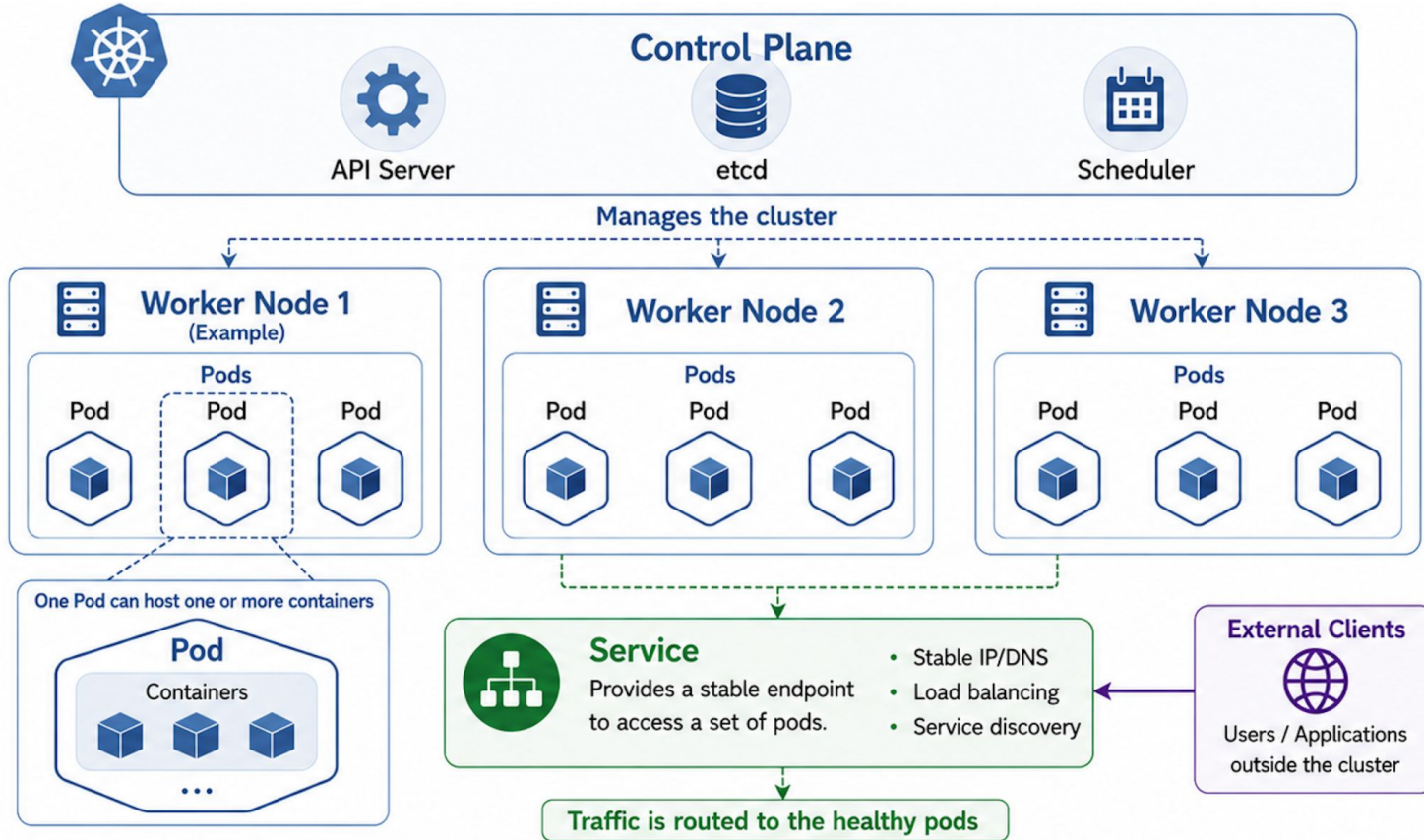


Core capabilities

- **Automated** rollout
- **Self-healing**
- **Scaling** support
- **Resource** abstraction
- **Smarter** scheduling



Kubernetes: A High-Level Architecture



How Kubernetes Organizes Resources



Kubernetes Cluster



Worker Node 1

Namespace 1



Pod



Service



ConfigMap



Secret



Deployment



Job / CronJob



PVC

Namespace 2



Pod



Service



ConfigMap



Secret



Deployment



Job / CronJob



PVC

The StatefulSet: Kubernetes' Answer to Databases

1. WHY REGULAR PODS ARE NOT ENOUGH



Kubernetes was built for stateless workloads

Stateless apps can be stopped and rescheduled easily.



Containers are ephemeral

If a pod is recreated, its container and IP can change.



2. WHAT DATABASES NEED



Stable identity

Consistent network identity across restarts.



Persistent storage

Data must survive pod restarts and failures.



Ordered lifecycle

Create, update, and stop pods in a defined order.

3. WHAT STATEFULSET PROVIDES

1



Stable, unique network identity

Each pod gets a persistent DNS-based hostname.

2



Persistent storage

Stable volumes stay with the pod across restarts.

3



Ordered deployment & scaling

Pods are created and updated in the required order.

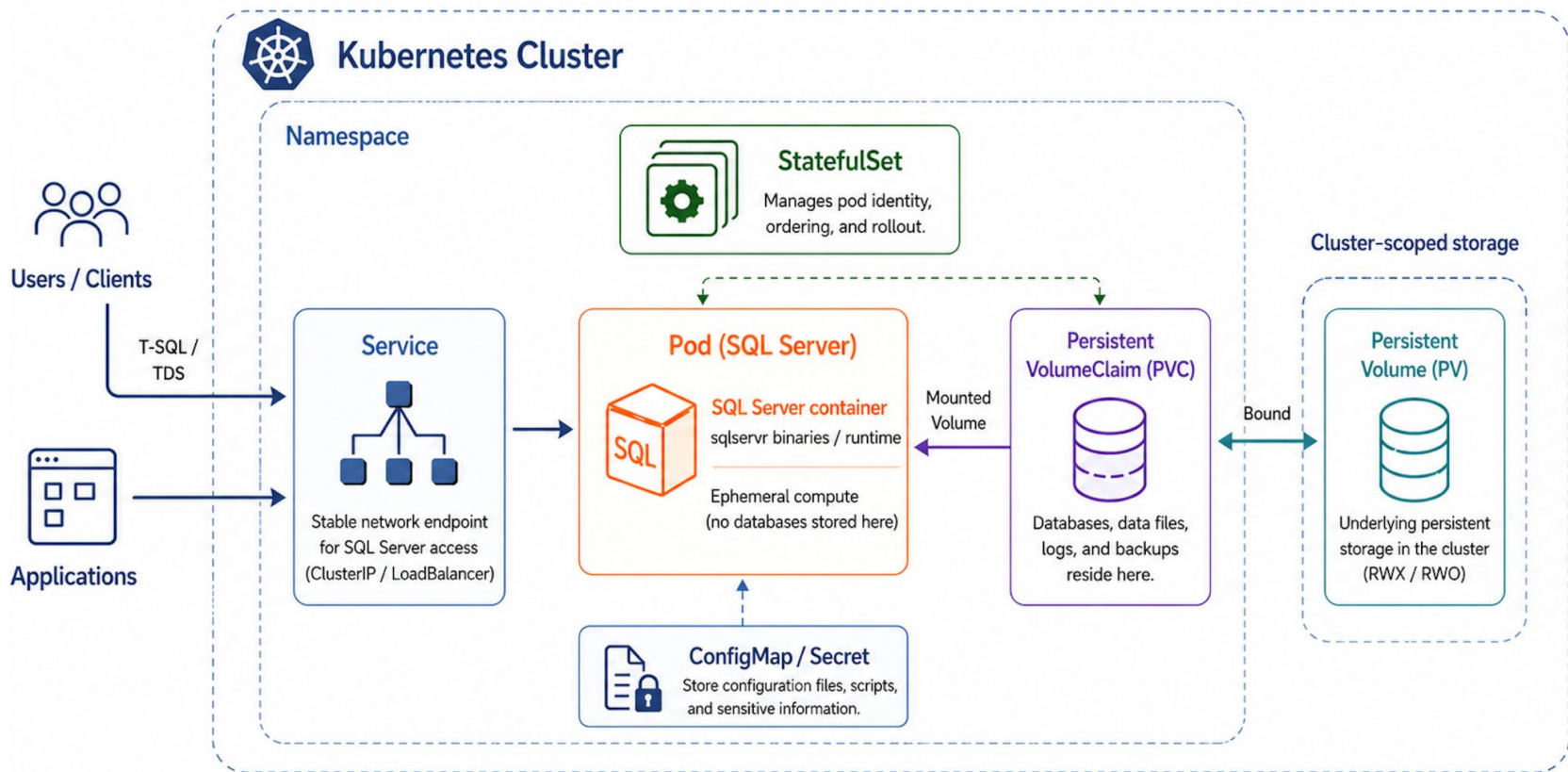
4



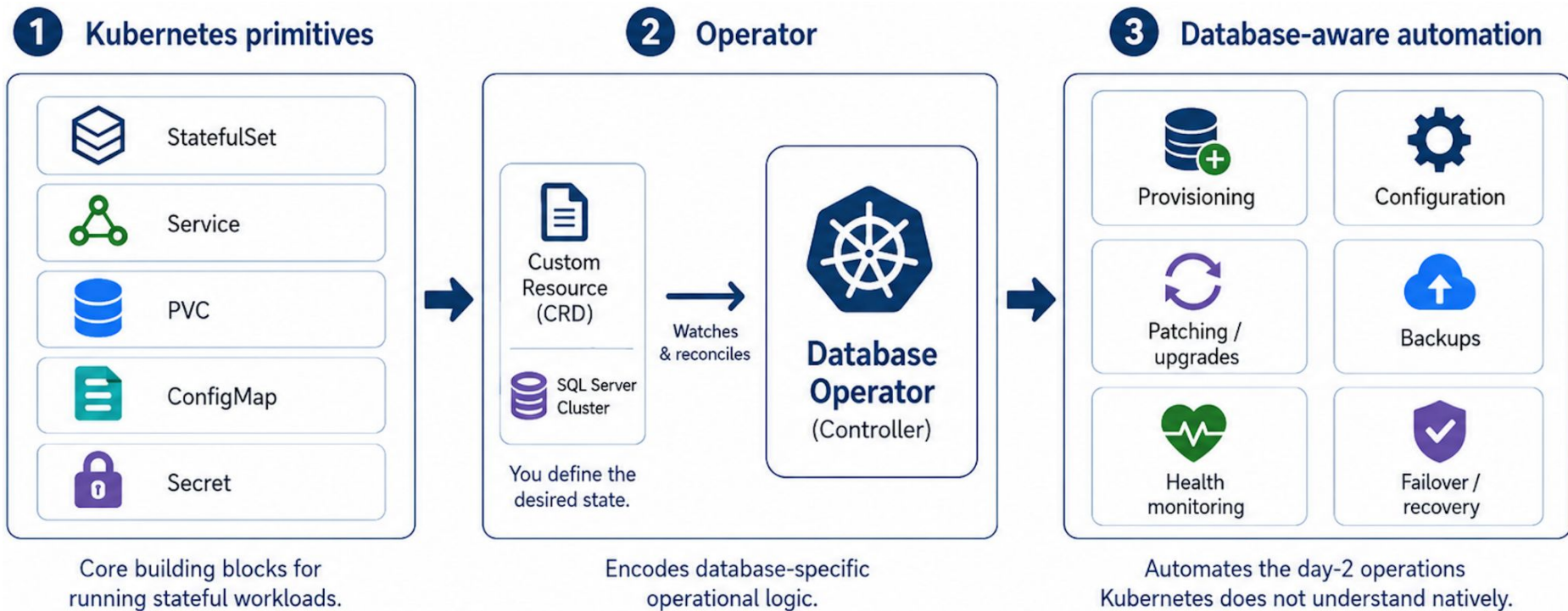
Ordered termination & scale-down

Pods shut down in the reverse order.

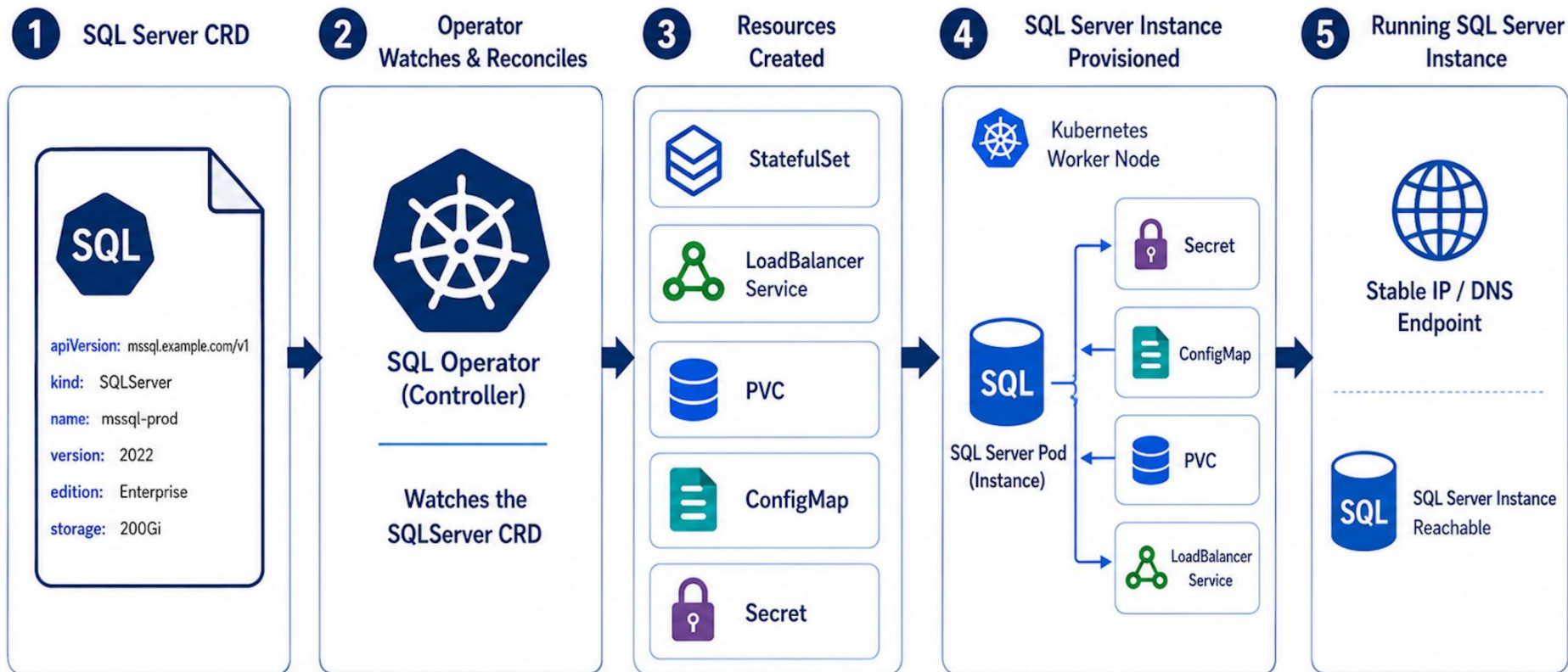
Building Blocks: Key Resources for SQL Server on K8s



Why Operators Matter for Stateful Databases



Operator-Driven SQL Server Deployment on Kubernetes



SQL Server High Availability on Kubernetes



When SQL Server fails,
what exactly should recover?



Service availability

- Keep the SQL endpoint reachable
- Restore access quickly after failure
- Usually handled by platform-level recovery



Data availability

- Keep the database role available
- Preserve access to data safely
- Requires database-aware failover logic



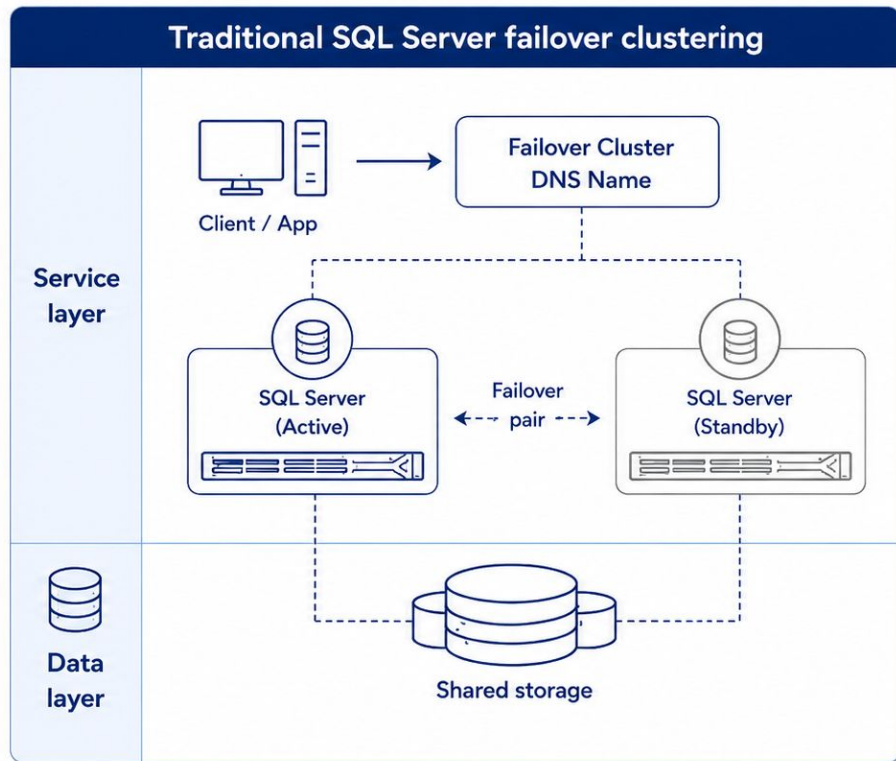
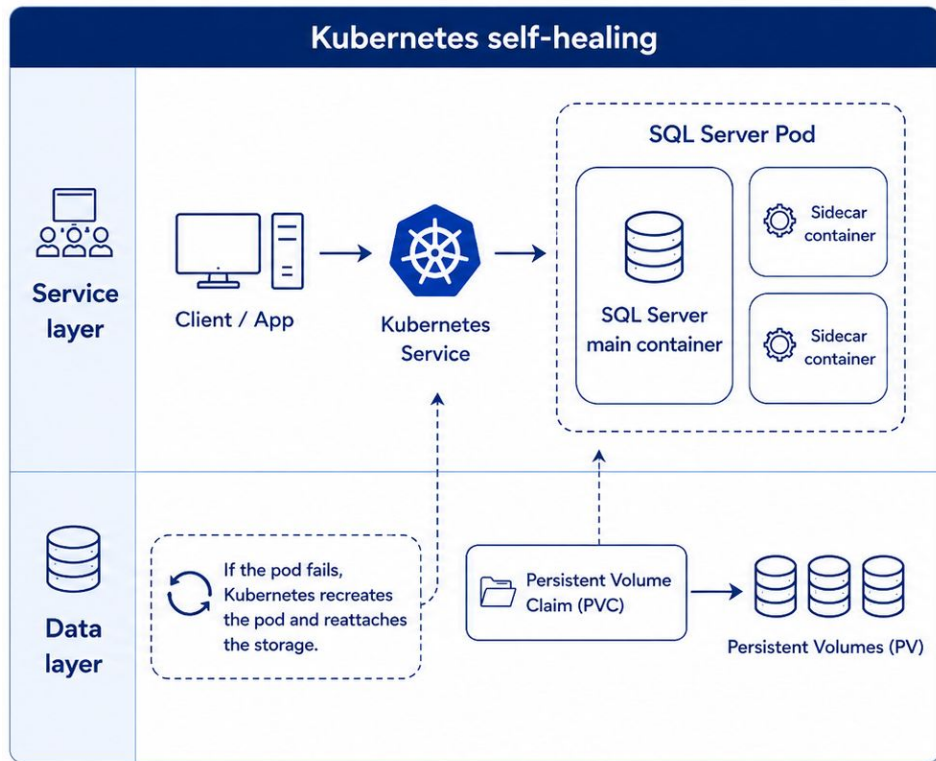
Kubernetes self-healing

..... or



Availability Groups +
cluster manager

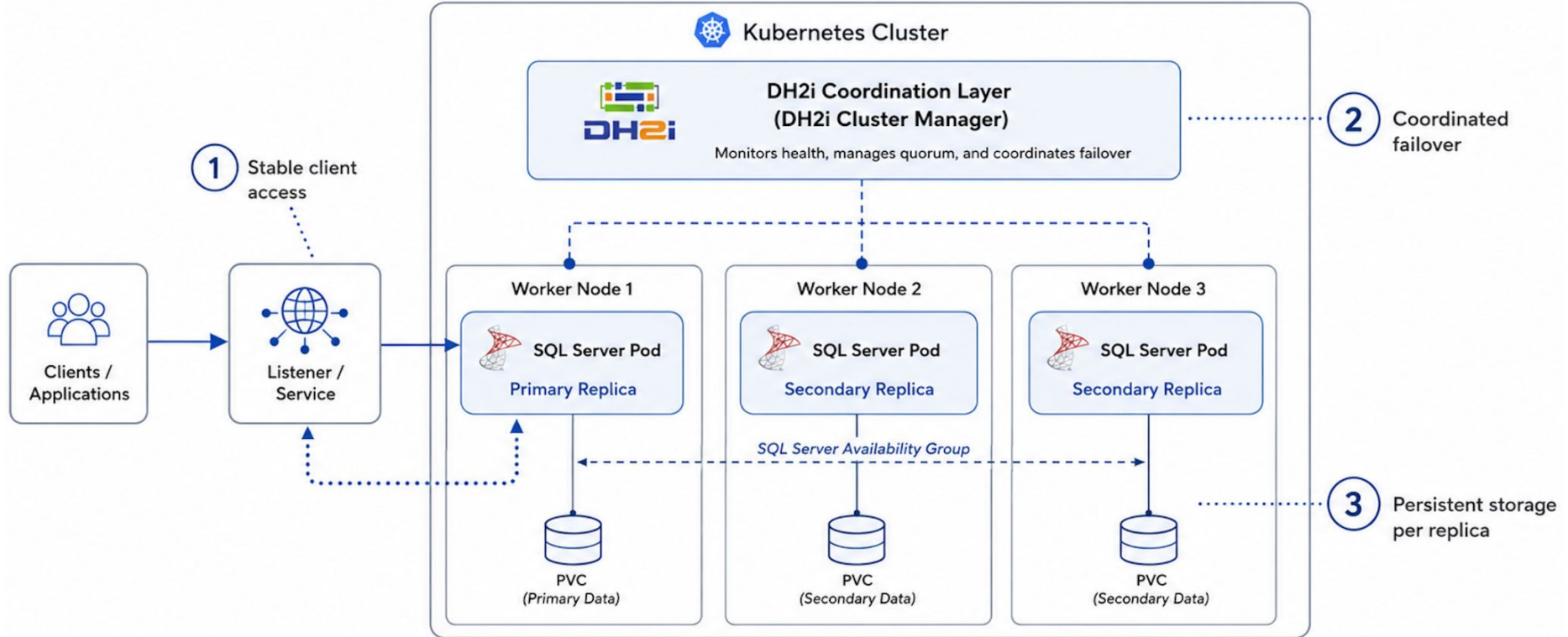
Kubernetes Self-Healing vs. SQL Server Failover Clustering



Cluster Manager for Availability Groups on Kubernetes



SQL Server Availability Group Architecture on Kubernetes



Key Takeaways

1



Containers modernize SQL Server delivery

Faster provisioning, consistency, and portability.

2



Kubernetes provides the orchestration layer

It adds scheduling, self-healing, and operational structure.

3



Stateful workloads need more than generic primitives

StatefulSets, storage, services, and operators make databases workable on Kubernetes.

4



Self-healing is not the whole HA story

Real database HA still needs Availability Groups and failover coordination.

5



A complete SQL Server HA design combines multiple layers

Kubernetes for runtime, AGs for replication, and cluster-manager logic for coordinated failover.



Kubernetes can run SQL Server well — but real success comes from combining cloud-native orchestration with database-aware design.

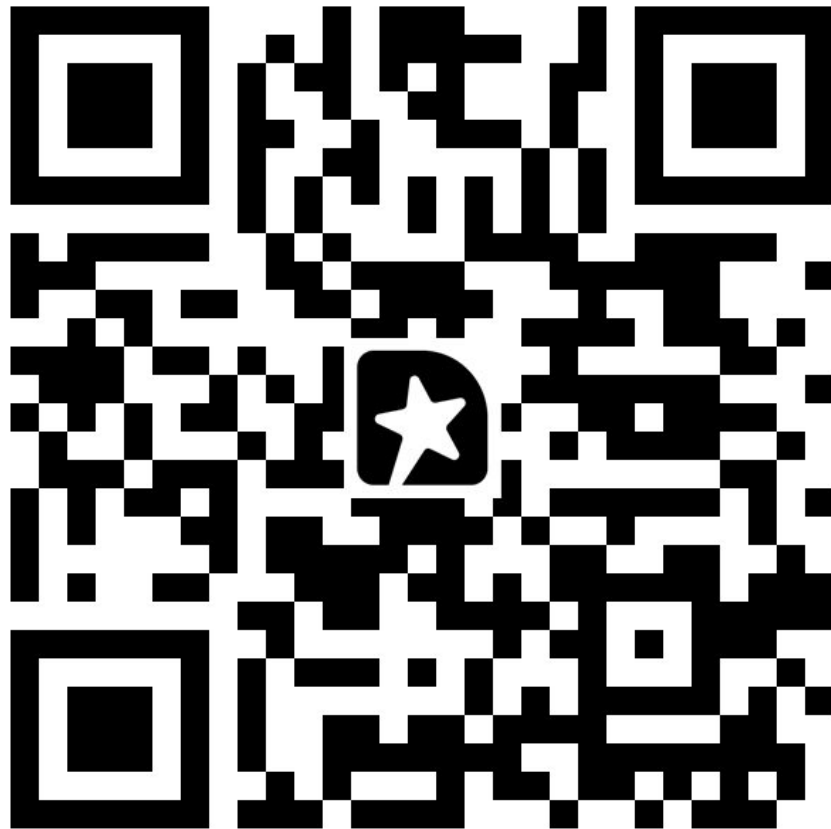


Demo + Questions

Let's see it in action.

Go raibh maith
agaibh

Thank You



SQL Server on Kubernetes: Redefining Modern Database
Infrastructure